

Conseal — Problem 3: Fixing the Tool's Mistakes

A PII correction interface for reviewers who move fast and trust automated suggestions slightly too much — the person who catches obvious errors but lets subtle ones slip by on autopilot.

The core tension

The failure mode isn't that Sam doesn't care. It's that he reviews quickly, and the tool's mistakes are distributed unevenly: most suggestions are right, which trains him to approve without reading. Uniform friction — a confirmation click on every span — would make him abandon the tool or click through mindlessly, which is worse. The only lever that works is *selective* friction: no interruption when he's in a good rhythm, a pointed nudge exactly when behavioral signals suggest he's stopped actually reading. This is the design principle every feature below flows from.

Four features built from that tension

- **Boundary adjuster:** the tool's existence-call can be right while its boundary is off by a character. Drag handles on span edges let Sam fix a clipped email address or a trailing digit without retyping — removing the friction that makes reviewers accept bad boundaries rather than correct them.
- **Consistency engine:** when Sam redacts one occurrence of "Daniel Okafor" and leaves two others visible, he gets a targeted alert listing exactly those two spans — catching the case where he reviewed the first occurrence carefully and rubber-stamped the rest.
- **Pattern matcher:** a regex pass over the document finds PII-shaped strings the model didn't flag (a second phone number in dot-delimited format, an SSN-shaped ID). These surface as pending review, not auto-redacted — keeping Sam's judgment in the loop rather than silently fixing the model's miss.
- **Export gate:** before download, a forced diff screen shows every span changed from the original suggestion and lists anything still unresolved. Sam must explicitly confirm he has reviewed outstanding items before export is allowed. This is the last possible save point before a mistake becomes real.

What I deliberately did not build

1. No custom-trained PII model: I used a cloud LLM for detection specifically because it produces realistic, messy output — false positives, missed entities, off-by-one boundaries — which is the raw material the correction experience is designed around. A fixed mock list would have made the demo look cleaner and taught nothing about real reviewer behavior.
2. No auth, multi-user, or team features: single-session prototype. Adding auth would have

consumed time better spent on the correction workflow itself.

3. No batch or multi-document review: that is explicitly Problem 2 (Maya). One document at a time was a deliberate scope decision, not a gap.
4. No ML-based rapid-action detection: a simple timing heuristic — three corrections within two seconds triggers a nudge — is the right amount of engineering for this signal. A learned classifier would require training data, add latency, and provide no meaningful improvement over a clock.

One honest limitation

Fuzzy name matching is naive: the consistency engine groups "Daniel Okafor" and "Mr. Okafor" only if the shorter form shares tokens with the full name. A different phrasing — "the patient," a middle name, a misspelling — breaks the grouping silently, which is exactly the kind of miss this tool is supposed to prevent. With more time I would add proper entity normalization, or at minimum surface unmatched name-shaped tokens as candidates rather than ignoring them.
